

M2 Sciences de l'Image

Moteur Physique

Projet 2026

à rendre avant le 30 Avril 2026
sur eLearning ou (eric.incerti@univ-eiffel.fr)

Sur la base des systèmes masse-ressort 2D vus en TD, plusieurs développements peuvent être envisagés, en prolongement de certaines propriétés et/ou défauts de ces modèles.

Base minimale : Système Masses-Ressorts en dimension 3 - le drapeau

Le modèle du drapeau flottant dans le vent est un des plus simples à mettre en oeuvre et permet de vérifier la validité et la fiabilité des modèles implantés.

Choix de la résolution (dimensions ($n \times p$) du drapeau) :

Elle doit être suffisante pour permettre des déformations riches mais rester limitée pour autoriser des jeux de paramètres (k, z, Fe) ne provoquant pas de divergence [cf. CM p.60-70]

☞ $n \in [10, 50]$ est un choix raisonnable.

Topologie de connexion :

C'est la question la plus importante. Cette construction fait écho à des considérations très classique en mécanique des structures : la notion de [tenseurs de contraintes](#).

Ces outils mathématiques (matriciels) permettent de décrire de façon infinitésimale la circulation des forces appliquées en un point d'un objet tridimensionnel (théorie des poutres). On s'intéresse seulement aux trois principaux : le tenseur de déformation, le tenseur de torsion et le tenseur de courbure.

Ici, nous ne procédons pas par intégration de contraintes locales, mais par assemblage de modules élémentaires : chaque *tenseur* se traduit alors par un sous-réseau de connexions spécifique.

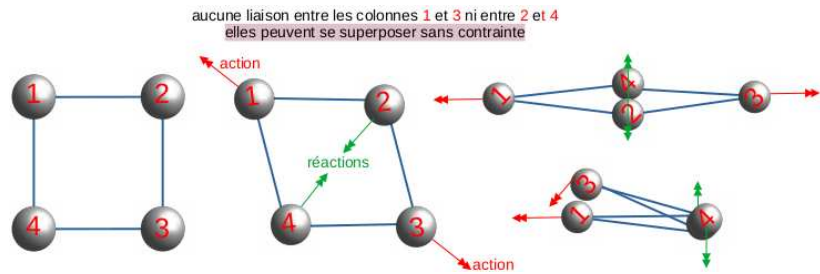
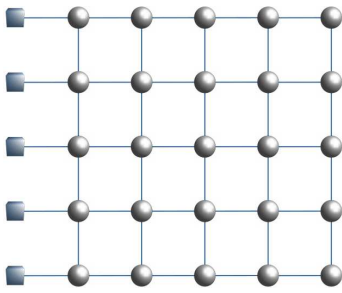
Nous allons voir que, pour modéliser un objet possédant une structure bi-/tri-dimensionnelle physiquement cohérente, ces 3 éléments sont nécessaires.

① Tenseur/Maillage de déformation

C'est le maillage principal, celui qui donne la forme et le volume global de l'objet et qui absorbe les contraintes unidirectionnelles et uniformes. Il correspond très exactement au modèle mécanique de *tenseur de déformation* (ou *tenseur de structure*)

Il est néanmoins insuffisant pour assurer une résistance aux contraintes multidirectionnelles (opposées). En particulier, si il suffit pour définir la "forme initiale" d'un objet carré, il est incapable d'en assurer la structure comme le montre la figure suivante (les masse 1 et 3 son indépendantes et peuvent se superposer, de même que les masses 2 et 4).

Pour assurer cette résistance à des contraintes de *torsion*, il faut une seconde couche de maillage.

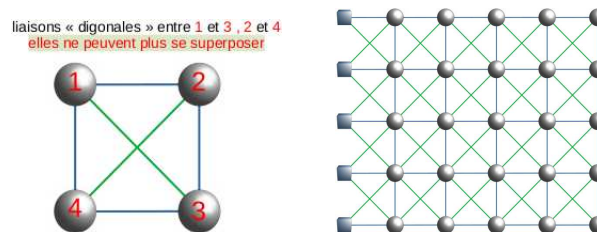


② Tenseur/Maillage de torsion

Ce maillage diagonal permet de rigidifier la structure. Quatres masses peuvent maintenant définir un "vrai" carré.

Il en va de même en dimension 3. Les liens diagonaux peuvent être définis directement sur les 6 faces ("héritage" du carré 2D) ou mieux sur les 4 diagonales du cube lui même (plus économique et beaucoup plus efficace).

Ce maillage s'assimile au modèle mécanique de *tenseur de torsion*.



Il est néanmoins toujours insuffisant pour assurer une résistance à la courbure.

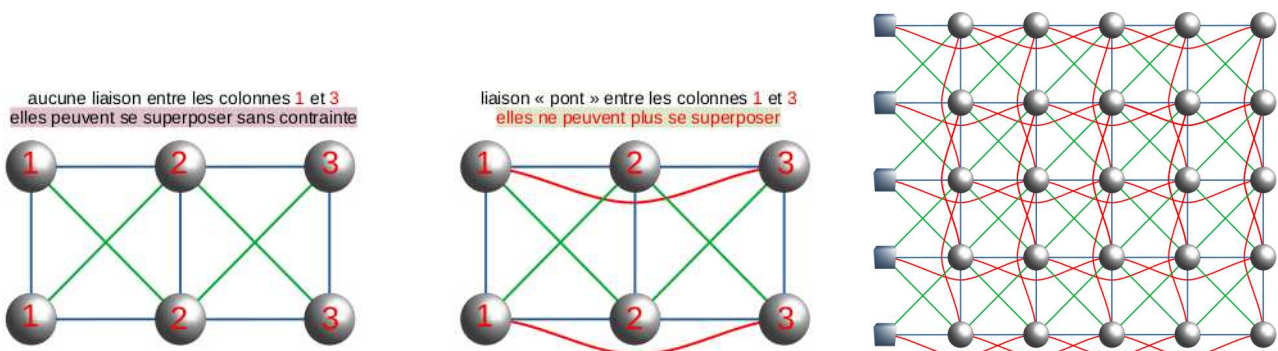
③ Tenseur/Maillage de courbure

Cette troisième couche de maillage relie des masses qui ne sont pas directement "voisines" (au sens géométrique).

Dans l'exemple ci-dessous, à gauche, 2 carrés reliés peuvent se "replier" en un seul (les colonnes 1 et 3 peuvent se superposer sans contrainte). Le drapeau ainsi maillé résiste bien à la torsion mais se replie comme un accordéon.

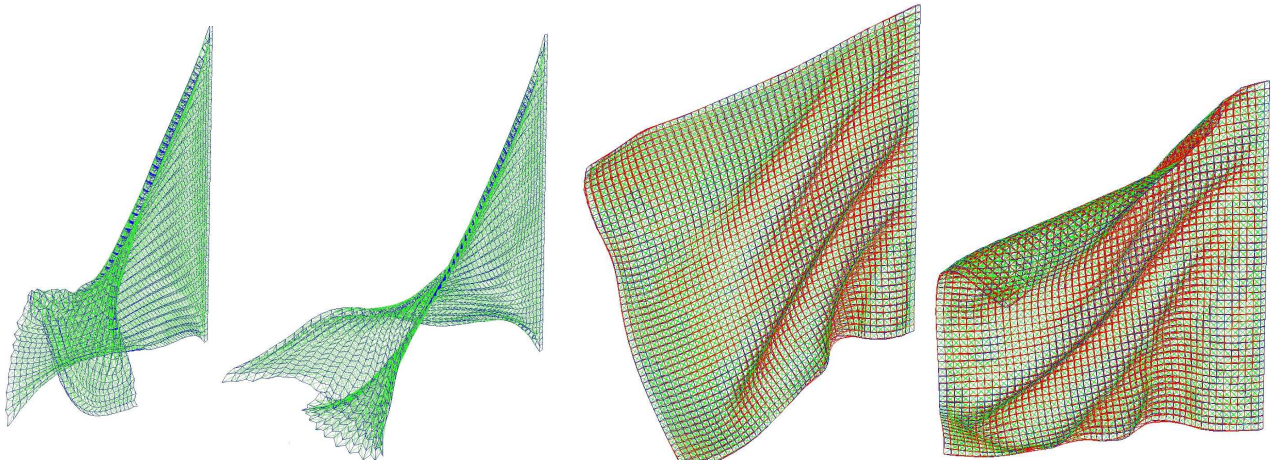
L'introduction d'un maillage "pont" entre les colonnes 1 et 3 résoud ce problème et confère à l'ensemble une bonne résistance à la courbure.

Ce maillage s'assimile au modèle mécanique de *tenseur de courbure*.



La modélisation par "assemblage" rejoint donc la mécanique du continu pour ce qui est de la construction de structures dynamiques rigides et il est assez simple de montrer que les équations définies par ces trois maillages sont en tout point identiques (dans un cadre simplifié) à celles définies par la mécanique des tenseurs.

On peut bien sûr imaginer toutes sortes de maillages (homogènes ou non, réguliers ou non...) mais ces trois contraintes doivent de toute façon être couvertes.



un drapeau 50x50 sans (gauche) et avec (droite) le maillage de courbure

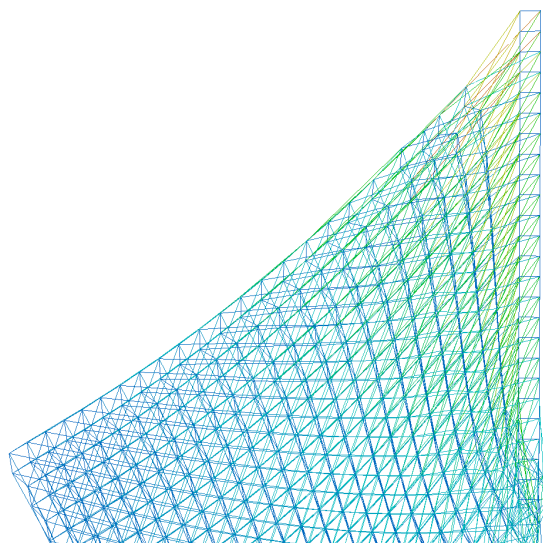
Paramétrage :

A priori il est sage de commencer avec un paramétrage homogène : toutes les masses à $M_a = m = 1$, toutes les liaisons identiques ($k = K_a * F_e^2$ et $z = Z_a * F_e$).

Effet "super-élastique" :

On peut observer un défaut classique de ces modèles connu sous le nom de "super-élasticité" : l'objet se comporte plus comme du caoutchouc que comme du tissu. Cet effet est particulièrement visible là où les contraintes sont maximales : les liaisons supportent tout le poids du modèle et sont donc particulièrement distendues.

Dans le cas du drapeau, c'est au niveau des points fixes les plus hauts que cela s'observe.



Pour corriger (un peu) cet effet, on peut essayer un paramétrage non homogène en introduisant un coefficient réducteur, appliqué sur les masses et les liaisons :

- au niveau du "mât" ce coefficient vaut 100%.
- à l'autre extrémité, il vaut, par exemple, 10%

L'extrémité "flottante" du drapeau est ainsi moins lourde et plus souple, les mouvements sont plus fluides et plus réalistes.

Le modèle de "vent" :

Pour animer le drapeau, il faut bien sûr des contraintes extérieures.

- La gravité : simple force verticale constante, s'appliquant sur toutes les particules.
- Le vent : force externe (comme la gravité) mais variable en direction et en intensité.

Un modèle tel que $\vec{W}(t) = \begin{pmatrix} a_x * \cos(f_x * t) \\ a_y * \cos(f_y * t) \\ a_z * \cos(f_z * t) \end{pmatrix}$ suffit, ($\vec{a}$ et $\vec{f}$ contrôlent l'amplitude et la fréquence).

Ce modèle de vent peut sans problème intégrer la gravité.

 il s'agit là d'un modèle descriptif ultra simpliste : si on applique un «vent» constant (force/direction, comme un ventilateur), le drapeau se figera très vite dans une position stable, ce qui ne se produirait pas dans la réalité physique.

Pour avoir un modèle un peu plus «physique» il faudrait appliquer cette force externe sur des portions de surface orientées (i.e. caractérisées par des normales) et gérer les occlusions, un peu comme dans un modèle d'illumination de type RayTracer (le vecteur «vent» étant le «rayon»).

Prolongation - thème 1 : gestion des auto-collisions

Le modèle de drapeau précédent ne gère pas les auto-collisions : des portions éloignées de la structure peuvent se "traverser" sans contrainte lors des mouvements.

En terme de complexité, un système **masse-ressort** à topologie fixe tel que le modèle précédent est de complexité linéaire par rapport au nombre de particules (moyennant un coefficient multiplicatif sur le nombre de liaisons, en fonction de la topologie de maillage).

Dès que l'on s'intéresse aux auto-collisions, les choses se compliquent puisque, dans l'absolu, chaque particule libre peut venir "heurter" n'importe quelle autre particule (libre ou fixe). On passe donc à un système de complexité quadratique par rapport au nombre de particules libres.

Il devient alors nécessaire de recourir à des méthodes d'optimisation. On trouve de nombreux exemples dans la littérature, mais la plupart sont basées sur des techniques de type "bounding box" permettant de déterminer les zones d'autocollisions.

La version la plus simple consiste à construire une grille (régulière ou hiérarchique) sur l'espace dans lequel évolue le modèle : à chaque instant on connaît la distribution des particules dans les cellules de la grille et on effectue les calculs d'auto-collision cellule par cellule et sur des cellules voisines. On obtient donc une complexité quadratique sur un nombre beaucoup plus réduit de particules.

Une difficulté est d'adapter la résolution de la grille pour dimensionner au mieux le nombre P de cellules contenant des particules et le nombre moyen n de particules par cellule pour optimiser la complexité moyenne (de l'ordre de $P * n^2$).

Une autre difficulté est d'éviter de lancer des calculs d'auto-collision entre des particules liées topologiquement (ie. par le maillage du modèle). Une solution envisageable est de calculer, pour chaque cellule, une *signature* basée sur la *distance topologique* des particules qu'elle contient : deux particules topologiquement voisines apportent une contribution nulle à cette signature alors que 2 masses éloignées dans la structure du drapeau apportent une contribution plus importante.

Cette signature peut se calculer assez simplement lors de la phase de mise-à-jour des états (intégrations numériques), en même temps que la distribution des particules dans les cellules : il suffit pour cela de connaître (calculer ou stocker) les indices topologiques des particules dans le maillage du drapeau. Ainsi, seules les cellules possédant une signature *forte* déclencheront les calculs d'auto-collision.

Un autre méthode, plus simple, consiste à utiliser une matrice booléenne $\mathcal{B}_{(N \times N)}$, assez similaire à la matrice de connexion du réseau (symétrique réelle), précisant si 2 particules sont susceptibles d'entrer en collision ou non. Pour deux particules p_i et p_j se trouvant dans la même cellule et telles que $\mathcal{B}(i,j) = 0$ il faut tester la collision.

Là encore, la contrainte à respecter est que la gestion des cellules et les calculs d'auto-collision aient le moins d'impact possible sur le moteur de simulation.

Prolongation - thème 2 : collisions sur points fixes géométriques

Il existe essentiellement deux méthodes pour la gestion des collisions, chacune ayant ses avantages et ses inconvénients les deux pouvant coexister dans un même modèle.

Par liaison conditionnelle

Cette méthode est simple et pratique pour gérer les auto-collisions et collisions entre deux objets mobiles (ce qui revient au même). Elle utilise des liaisons conditionnelles de type *ressort-frein* seuillé actives (production de force) uniquement lorsque deux particules sont proches (distance inférieure à un seuil, caractéristique de la liaison).

Avantages

Les principaux avantages de cette méthode sont sa simplicité et sa parfaite adéquation avec les principes de modularité et de généricité des systèmes *masse/ressort*. N'importe quel type de liaison peut, moyennant un seuillage sur la distance, servir à modéliser un choc entre points matériels.

Inconvénients

Les défauts sont malheureusement plus nombreux que les avantages. En premier lieu, le principe même des systèmes *masse/ressort* fait que les objets matériels sont ponctuels et que la géométrie d'une scène n'est portée que par la connexion de liaisons entre ces points et des calculs de distance. Ainsi, la distance euclidienne classique ne permet de modéliser que des objets sphériques. Heureusement on peut surmonter cette limitation (cf. plus loin).

De plus, avec des liaisons visco-élastiques, il est quasiment impossible de modéliser des objets rigides : la collision étant en fait un *choc élastique*, les objets restent en contact et se rapprochent jusqu'à ce

que la force produite par le ressort les repousse au delà du seuil de collision. Ainsi, durant ce temps de contact, les deux points ont tendance à s'inter-pénétrer.

Enfin, le modèle de base de frottement étant le *frein cinétique* (force de frottement proportionnelle à la vitesse relative des points), il est très difficile de simuler des contacts prolongés. En effet, plus la vitesse relative diminue, moins le frein dissipe d'énergie et les objets ont tendance à glisser lentement l'un sur l'autre.

Par dynamique inverse

Cette méthode est beaucoup plus intéressante pour gérer les contacts prolongés et les glissements sur une surface plane, à condition que cette surface soit fixe (on peut également l'appliquer à des surfaces mobiles, mais c'est beaucoup plus délicat). Elle permet donc de définir des obstacles de forme quelconque du moment qu'ils sont facétisés.

Principe de fonctionnement

Une facette est caractérisée par ses sommets (A,B,C) qui délimitent une zone du plan et par la normale $\vec{N}$ à ce plan.

① Détection (détermination du point d'impact) :

Se fait par un algorithme très simple de type *intersection rayon-facette*, classique en infographie, adapté pour la circonstance. Le rayon est ici donné par la position courante $P(t)$ de la masse libre et son vecteur vitesse courant $\vec{V}(t)$.

L'algorithme commence par calculer la position qu'aurait le point à l'instant suivant en l'absence de collision et de toute autre force active, soit $Q(t) = P(t) + h \times \vec{V}(t)$.

On teste ensuite si le segment $[P(t), Q(t)]$ intersecte la facette. Pour qu'une collision entre la masse libre et la facette ait lieu entre les instants (t) et $(t + h)$, il faut tout d'abord vérifier que la masse est bien *au-dessus de la facette* et que sa direction de déplacement est bonne. Pour cela, si on note $\vec{N}$ la normale (constante) à la facette, il suffit de vérifier que la double inégalité suivante :

$$\begin{cases} \vec{N} \bullet \overrightarrow{AP(t)} > 0. & \text{la masse est au dessus du plan de la facette} & (a) \\ \vec{N} \bullet \overrightarrow{PQ(t)} < 0. & \text{la masse se dirige vers la facette} & (b) \end{cases}$$

En pratique, pour l'inéquation (a), on vérifiera plutôt : $(\vec{N} \bullet \overrightarrow{AP(t)} > \epsilon)$, où ϵ est une valeur strictement positive faible (10^{-6} par exemple). Cela permet d'éviter la plupart des cas d'interpénétration.

Si l'une des deux inéquations (a) et (b) n'est pas vérifiée, l'algorithme s'arrête. Sinon cela signifie que la demi-droite $\overrightarrow{PQ(t)}$ traverse le plan de la facette en un point M à déterminer.

Pour conclure qu'il y a collision, il faut encore vérifier deux points :

1. Le point M doit se trouver entre $P(t)$ et $Q(t)$.

Pour cela, on forme le rapport $u = -\frac{\vec{N} \bullet \overrightarrow{AP(t)}}{\vec{N} \bullet \overrightarrow{PQ(t)}}$ et on obtient le point $M = P(t) + u \times \overrightarrow{PQ(t)}$.

Puisque l'on sait déjà que $(u > 0)$, il suffit de vérifier que $(u \leq 1)$. Si ce n'est pas le cas, c'est que $Q(t)$ est aussi au-dessus de la facette. Il n'y donc pas de collision dans cet intervalle de temps et l'algorithme s'arrête.

- Si le point précédent est validé, il reste à vérifier que le point M trouvé appartient bien à la facette associée au point fixe. Il suffit de vérifier que les trois produits vectoriels $(\vec{MA} \wedge \vec{MB})$, $(\vec{MB} \wedge \vec{MC})$ et $(\vec{MC} \wedge \vec{MA})$ ont même orientation. Si ce n'est pas le cas, alors M est à l'extérieur de la facette et l'algorithme s'arrête.

Si tous les tests précédents ont été passés avec succès, on peut alors conclure qu'une collision est prévisible. On met alors en place la réponse.

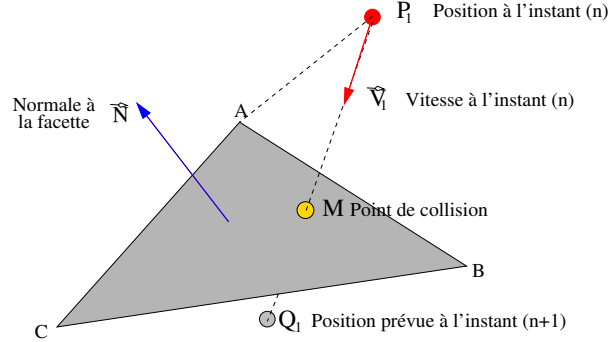


Figure 1: *détection de collision*

② Traitement par dynamique inverse (calcul des forces de réaction) :

Dans cette version, lorsque l'on a détecté une future collision, on traite la réponse non pas avec la position M du point d'impact trouvé, mais avec la position courante de la masse libre (là encore c'est pour limiter les inter-pénétrations).

- On commence par récupérer la force $\vec{F}(t)$ stockée dans le buffer de force de la masse libre, et on la décompose, par rapport au plan de la facette, en une composante normale $\vec{F}_N = (\vec{F}(t) \cdot \vec{N})\vec{N}$ et une composante tangentielle $\vec{F}_T = \vec{F}(t) - \vec{F}_N$.
- On fait de même pour la vitesse de la masse libre : $\vec{V}_N = (\vec{V}(t) \cdot \vec{N})\vec{N}$ et $\vec{V}_T = \vec{V}(t) - \vec{V}_N$.
- Ici interviennent deux paramètres *physique* associés à la facette : un coefficient de *dissipation* α et un coefficient de *frottement* β . Le premier exprime la quantité d'énergie perdue lors du choc, c'est-à-dire la *dureté* de l'obstacle, le second exprime la résistance au glissement, c'est-à-dire la *rugosité* de l'obstacle ($(\alpha, \beta) \in [0, 1]^2$).

La réaction à la collision s'exprime alors de la manière suivante :

$$\vec{F}'(t) = \begin{cases} -\alpha \vec{F}_N & \text{si } \|\vec{F}_T\| < \beta \|\vec{F}_N\| \quad (\text{rebond sans glissement}) \\ -\alpha \vec{F}_N + \left(1 - \beta \frac{\|\vec{F}_N\|}{\|\vec{F}_T\|}\right) \vec{F}_T & \text{sinon} \quad (\text{rebond avec glissement}) \end{cases}$$

et de même :

$$\vec{V}'(t) = \begin{cases} -\alpha \vec{V}_N & \text{si } \|\vec{V}_T\| < \beta \|\vec{V}_N\| \quad (\text{rebond sans glissement}) \\ -\alpha \vec{V}_N + \left(1 - \beta \frac{\|\vec{V}_N\|}{\|\vec{V}_T\|}\right) \vec{V}_T & \text{sinon} \quad (\text{rebond avec glissement}) \end{cases}$$

Avantages

L'avantage principal est bien sûr la possibilité de modéliser des obstacles facétisés de forme quelconque qui se comporteront comme de vrais solides indéformables (il n'y a pas d'inter-pénétration).

Attention cependant pour les objets composés de plusieurs facettes : des fuites peuvent se produire aux jointures. Pour éviter cela il faut parfois tricher un peu...

Inconvénients

Un gros défaut de cette méthode est qu'elle n'est pas très *physique* puisque la détection se fait sur des critères purement géométriques et le traitement (totalement déconnecté de la collision) consiste à décrire *l'effet* de la collision et non à *intégrer* une description des *causes*.

Mais c'est surtout à un niveau purement algorithmique que se pose le principal problème. Avec la méthode par liaison conditionnelle, une liaison de contact est traitée comme n'importe quelle autre, de manière totalement transparente pour le moteur de simulation. Ce n'est plus le cas ici : le fait que le traitement de la collision consiste à *symétriser* le vecteur de force stocké dans la particule fait que ce calcul doit *impérativement* être fait *après* que toutes les liaisons classiques ont apporté leur contribution à ce vecteur et *avant* que la particule ne mette à jour sa position en intégrant l'équation de Newton (avec le nouveau vecteur de force).

Avec cette méthode, le moteur de simulation doit donc fonctionner en trois étapes :

- Ⓐ Calcul des forces produites par les liaisons physiques classiques.
- Ⓑ *Gestion des collisions* sur objets facétisés.
- Ⓒ Mise-à-jour des états des particules.

Liaison conditionnelle sur distance non euclidienne

Jusqu'à présent, lorsqu'il fallait calculer des distances, il s'agissait toujours de la distance associée à la *norme euclidienne* (ou *norme L2*) $d(A, B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2 + (z_B - z_A)^2}$.

Une liaison conditionnelle de seuil r branchée sur un point A délimite une *sphère* de rayon r autour de A . Il suffira donc de tracer cette sphère pour avoir l'illusion qu'elle existe physiquement en tant qu'objet.

1^o exemple : la distance cubique et transformations géométriques

Mais la norme euclidienne n'est pas la seule envisageable. On connaît par exemple la *norme du max* (ou *norme infinie*) définissant la distance $d(A, B) = \text{Max}(|x_B - x_A| + |y_B - y_A| + |z_B - z_A|)$.

Dans une liaison conditionnelle classique, si on remplace la distance euclidienne par celle-ci, le volume délimité⁽¹⁾ devient un *cube* de demi-côté r .

Le seul problème qui apparaît est que, si la distance euclidienne est isotrope (une sphère n'a pas d'orientation), ce n'est plus le cas ici : le cube défini par cette distance est *parallèle aux axes* du repère.

Pour en faire quelque chose d'intéressant, il faut au moins associer à cet objet la possibilité de l'orienter. Pour cela il faut pouvoir lui appliquer des rotations et donc lui associer des matrices de transformation directes et inverses. Dès lors on peut aussi lui appliquer les autres transformations classiques : homothéties et translation.

Avec ces transformations, la distance euclidienne classique permet de construire n'importe quel type d'ellipsoïde et la distance cubique n'importe quel type de parallélépipèdes et autres *losanges*.

⁽¹⁾ Boule Unité associée à la distance

Un nouvel objet matériel et une nouvelle liaison

Ainsi, on peut définir un nouvel objet dans notre environnement masse/ressort (ou particule) : le point fixe géométrique. C'est un point fixe comme les autres mais possédant en plus deux matrices de transformation qui lui confèrent un repère local et surtout une *fonction de distance* qui lui est propre et qui n'est pas nécessairement la distance euclidienne. L'utilisation de matrices de transformation permet en plus d'utiliser des *seuil canoniques* de valeurs 1.

Bien sûr il faut lui associer une liaison un peu spéciale et prendre quelques précautions : cette liaison ne peut être utilisée qu'entre un *point fixe géométrique* et une masse libre, elle n'est pas symétrique (chaque point de connexion doit respecter un type précis), et elle devra exécuter son algorithme (du *ressort/frein seuillé* classique) en utilisant la fonction de distance et les matrices de transformation propres au point fixe.

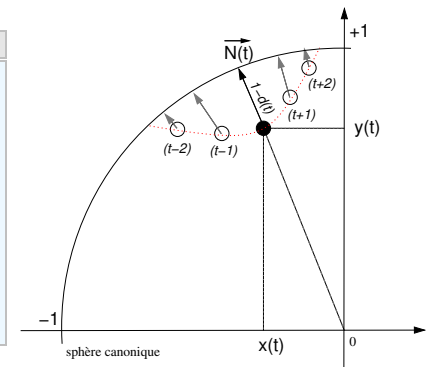
Structure de données

Ces nouveaux objets sont, du point de vue du simulateur, assimilables à des points matériels. Ils ont néanmoins une structure de donnée plus complexe.

- En tant que points fixes, ils n'ont pas besoin de vitesse ni d'accumulateur de force.
- En tant qu'objets volumiques il ont une position et une orientation : ces données sont gérées par des matrices de transformation classique (directe M_d , inverse M_i)
- Et surtout ils ont une forme (i.e. un *interieur* et un *exterieur*), définie par une référence canonique et une *fonction de distance* qui renvoie, sous la forme d'un vecteur, la distance et la direction de pénétration d'un point P à l'intérieur de l'objet canonique

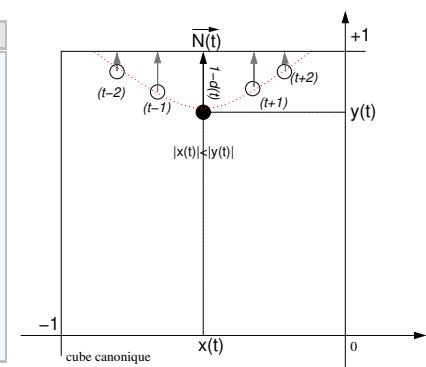
Par exemple, pour l'objet canonique <sphere>, la fonction de distance est définie par :

```
distance euclidienne
bool dist_sphere(Point P, Vecteur *N)
{
    double d = SQR(P.x)+SQR(P.y)+SQR(P.z); /* norme euclidienne */
    if (d>1.) return false;                /* le point P est dehors */
    d = 1./sqrt(d)-1.;
    N->x = P.x*d;
    N->y = P.y*d;
    N->z = P.z*d;
    return true;
}
```



De même, pour l'objet canonique <cube>, la fonction de distance se définit par :

```
distance euclidienne
bool dist_sphere(Point P, Vecteur *N)
{
    double d = d = MAX3(|P.x|,|P.y|,|P.z|); /* norme du max */
    if (d>1.) return false;                /* le point P est dehors */
    N->x=N->y=N->z=0.;
    if (d==|P.x|) { N->x = P.x*(1./d-1.); return true; }
    if (d==|P.y|) { N->y = P.y*(1./d-1.); return true; }
    N->z = P.z*(1./d-1.);
    return true;
}
```



Ainsi, dans ce modèle, la collision est gérée par une pénétration temporaire de la particule dans l'obstacle et non pas de manière instantanée comme dans le modèle par dynamique inverse. La

particule "entre" par un endroit, et "sort" par un autre. A chaque instant passé à l'intérieur de l'obstacle, un ressort-frein virtuel produit une force de répulsion proportionnelle, en intensité et direction, au *vecteur de pénétration canonique* $\vec{N}(t)$

Mise en œuvre

Le principe d'utilisation de ce nouveau *point matériel* est le suivant :

- ① **Modélisation** : on choisit la nature de la distance (sphere, cube...), les transformations qui définiront forme, position et orientation de l'objet géométrique souhaité (i.e. on construit ses deux matrices), on choisit la nature des interactions (ressort, frein...) de cet objet avec les particules libres.
- ② **Simulation** : l'algorithme associé aux liaisons spéciales est le suivant :
 - Ⓐ envoyer la particule (position et vitesse) dans le repère canonique du point fixe (via sa matrice de transformation inverse).
 - Ⓑ tester la position de ce point 'transformé' par rapport à l'objet canonique (avec la fonction de distance propre à l'objet)
 - Ⓒ Si le point a pénétré l'objet (collision détectée), la fonction renvoie le *vecteur de pénétration*.
 - Ⓐ calculer la *force de répulsion canonique* appliquée à la particule transformée en fonction des la nature et des paramètres de la liaison.
 - Ⓑ renvoyer cette force dans le repère global (via la matrice directe) et la sommer dans le *buffer de force* de la particule.

Avec ce principe on peut envisager assez simplement de construire d'autres objets tels que cylindre, tronc de cône, tore...

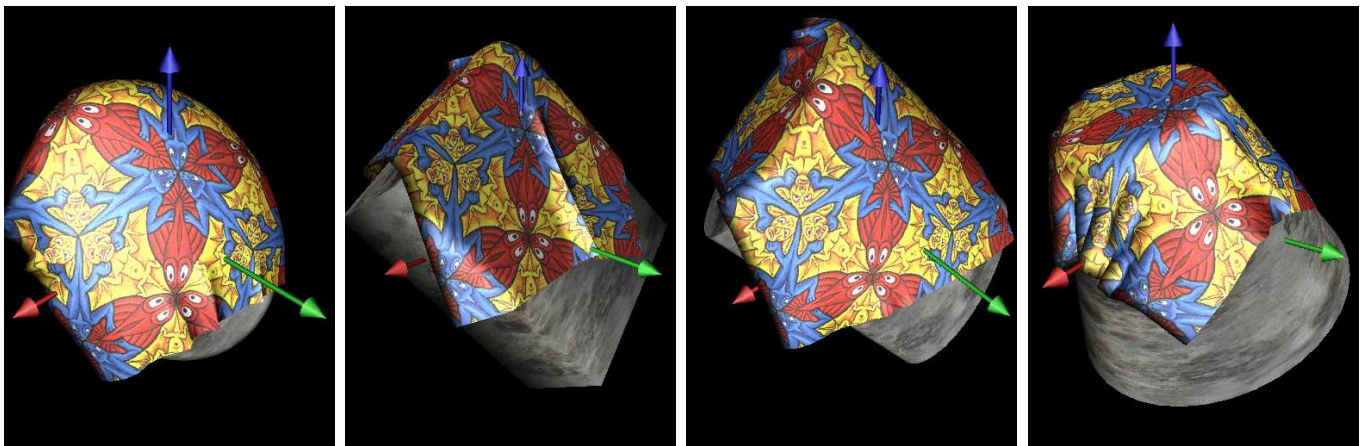


Figure 2: *distances spheriques, cubique, cylindrique et cônica*

Prolongation - thème 3 : rendu par Surfaces Implicites et GPU

Un autre sujet envisageable est l'utilisation d'un Modèle Physique limité (donc très dynamique, mais pas très beau) compensé par un moteur de rendu utilisant des surfaces implicites (Splines, NURBS...) s'appuyant sur le maillage physique pour interpoler les positions de l'objet entre les particules.

Ce dernier thème se marie très bien avec un portage total ou partiel sur GPU.

Travail à réaliser

Il s'agit, a priori, d'un travail individuel. Chaque étudiant doit choisir un des thèmes proposés (ou en proposer de nouveaux, ça n'est pas interdit...) et fournir un programme avec un maximum d'exemples d'illustration.

Même si rien n'est imposé quant au langage et interface graphique, vos programmes sont censés tourner sur les machines de l'université (Linux). Evitez donc si possible l'accumulation de bibliothèques exotiques et d'environnements spécialisés.

Dans la mesure du possible, un court rapport illustré de captures d'écran, ou mieux encore des vidéos, seront les bienvenus.

Les principaux critères d'évaluation concerneront les aspects "dynamiques" des simulations proposées (réalisme, performances, modularité). Le "rendu" est un problème différent qui ne nous intéresse pas réellement ici. Un affichage simple suffira. Pensez plutôt "visualisation scientifique" que "rendu" (à part bien sûr pour le thème 3).